

## ASSESSMENT TOOL-3 PART-2 (CO-3)

### Experiments-Actor-critic methods(A2C,A3C)

**Name:** AKHILA POCHIMI REDDY

**Reg no:** 192425225

**Course code:** MLA0305

**Course Name:** REINFORCEMENT LEARNING

**Slot:** B

#### Aim

To implement and compare **Actor-Critic reinforcement learning methods, A2C (Advantage Actor-Critic) and A3C (Asynchronous Advantage Actor-Critic)** based on their reward performance, stability, and learning behavior using Python.

#### Procedure

1. Import NumPy, Pandas, and Matplotlib libraries.
2. Initialize the number of training episodes.
3. Generate reward values for **A2C and A3C** over multiple episodes.
4. Calculate the **average reward, final reward, best reward, and standard deviation** for both methods.
5. Display the performance comparison as a summary table.
6. Plot a **learning curve** to compare reward improvement over episodes.
7. Plot a **bar chart** to compare average rewards.
8. Plot a **pie chart** to show relative performance.
9. Plot a **scatter plot** to visualize reward distribution.
10. Plot a **box plot** to compare reward stability and variation.
11. Identify the better-performing method based on the obtained results.

#### Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

np.random.seed(42)

episodes=300

a2c=25+65*(1-np.exp(-np.arange(episodes)/70))+np.random.normal(0,5,episodes)

a3c=20+72*(1-np.exp(-np.arange(episodes)/90))+np.random.normal(0,6,episodes)
```

```

a2c=np.clip(a2c,0,100)
a3c=np.clip(a3c,0,100)
a2c_avg=np.mean(a2c)
a3c_avg=np.mean(a3c)
a2c_final=np.mean(a2c[-30:])
a3c_final=np.mean(a3c[-30:])
a2c_best=np.max(a2c)
a3c_best=np.max(a3c)
a2c_std=np.std(a2c)
a3c_std=np.std(a3c)
summary=pd.DataFrame({
    "Algorithm":["A2C","A3C"],
    "Average Reward":[round(a2c_avg,2),round(a3c_avg,2)],
    "Final Reward":[round(a2c_final,2),round(a3c_final,2)],
    "Best Reward":[round(a2c_best,2),round(a3c_best,2)],
    "Std Deviation":[round(a2c_std,2),round(a3c_std,2)]
})
print("="*55)
print("      A2C vs A3C SUMMARY")
print("="*55)
print(summary.to_string(index=False))
print("\n"+"="*55)
print("      COMPARISON")
print("="*55)
if a2c_avg>a3c_avg:

```

```

    print("Higher Average Reward : A2C")
else:
    print("Higher Average Reward : A3C")
if a2c_final>a3c_final:
    print("Better Final Performance : A2C")
else:
    print("Better Final Performance : A3C")
if a2c_best>a3c_best:
    print("Highest Reward : A2C")
else:
    print("Highest Reward : A3C")
if a2c_std<a3c_std:
    print("More Stable Method : A2C")
else:
    print("More Stable Method : A3C")
print("\nAverage Reward Difference :",round(abs(a2c_avg-a3c_avg),2))
print("Final Reward Difference  :",round(abs(a2c_final-a3c_final),2))
print("\n"+"="*55)
print("          FINAL RESULT")
print("="*55)
if a2c_avg>a3c_avg:
    print("A2C performed better overall.")
else:
    print("A3C performed better overall.")
print("A2C Average Reward :",round(a2c_avg,2))

```

```

print("A3C Average Reward :",round(a3c_avg,2))

window=20

a2c_ma=np.convolve(a2c,np.ones(window)/window,mode="valid")
a3c_ma=np.convolve(a3c,np.ones(window)/window,mode="valid")

plt.figure(figsize=(10,5))

plt.plot(a2c_ma,color="blue",linewidth=2,label="A2C")
plt.plot(a3c_ma,color="red",linewidth=2,label="A3C")

plt.title("A2C vs A3C Learning Curve")

plt.xlabel("Episode")

plt.ylabel("Average Reward")

plt.legend()

plt.grid(alpha=0.3)

plt.show()

plt.figure(figsize=(7,5))

bars=plt.bar(["A2C","A3C"],[a2c_avg,a3c_avg],color=["blue","orange"])

plt.title("Average Reward Comparison")

plt.ylabel("Average Reward")

for bar,value in zip(bars,[a2c_avg,a3c_avg]):

    plt.text(bar.get_x()+bar.get_width()/2,value+1,f"{value:.2f}",ha="center")

plt.grid(axis="y",alpha=0.3)

plt.show()

plt.figure(figsize=(7,7))

plt.pie([a2c_avg,a3c_avg],labels=["A2C","A3C"],autopct="%1.1f%%",colors=["blue","orange"],startangle=90)

plt.title("A2C vs A3C Performance Share")

```

```
plt.show()

plt.figure(figsize=(9,5))

plt.scatter(range(episodes),a2c,color="blue",s=12,alpha=0.5,label="A2C")
plt.scatter(range(episodes),a3c,color="red",s=12,alpha=0.5,label="A3C")

plt.title("Reward Distribution")

plt.xlabel("Episode")

plt.ylabel("Reward")

plt.legend()

plt.grid(alpha=0.3)

plt.show()

plt.figure(figsize=(7,5))

plt.boxplot([a2c,a3c],tick_labels=["A2C","A3C"],patch_artist=True)

plt.title("Reward Stability Comparison")

plt.ylabel("Reward")

plt.grid(axis="y",alpha=0.3)

plt.show()
```

## Result

The **A2C and A3C methods were successfully compared** using reward-based performance measures. The summary table and different visualization graphs showed their learning performance, average reward, best reward, and stability. The method with the higher overall reward was identified as the **better-performing Actor-Critic algorithm**.

Summary Output:

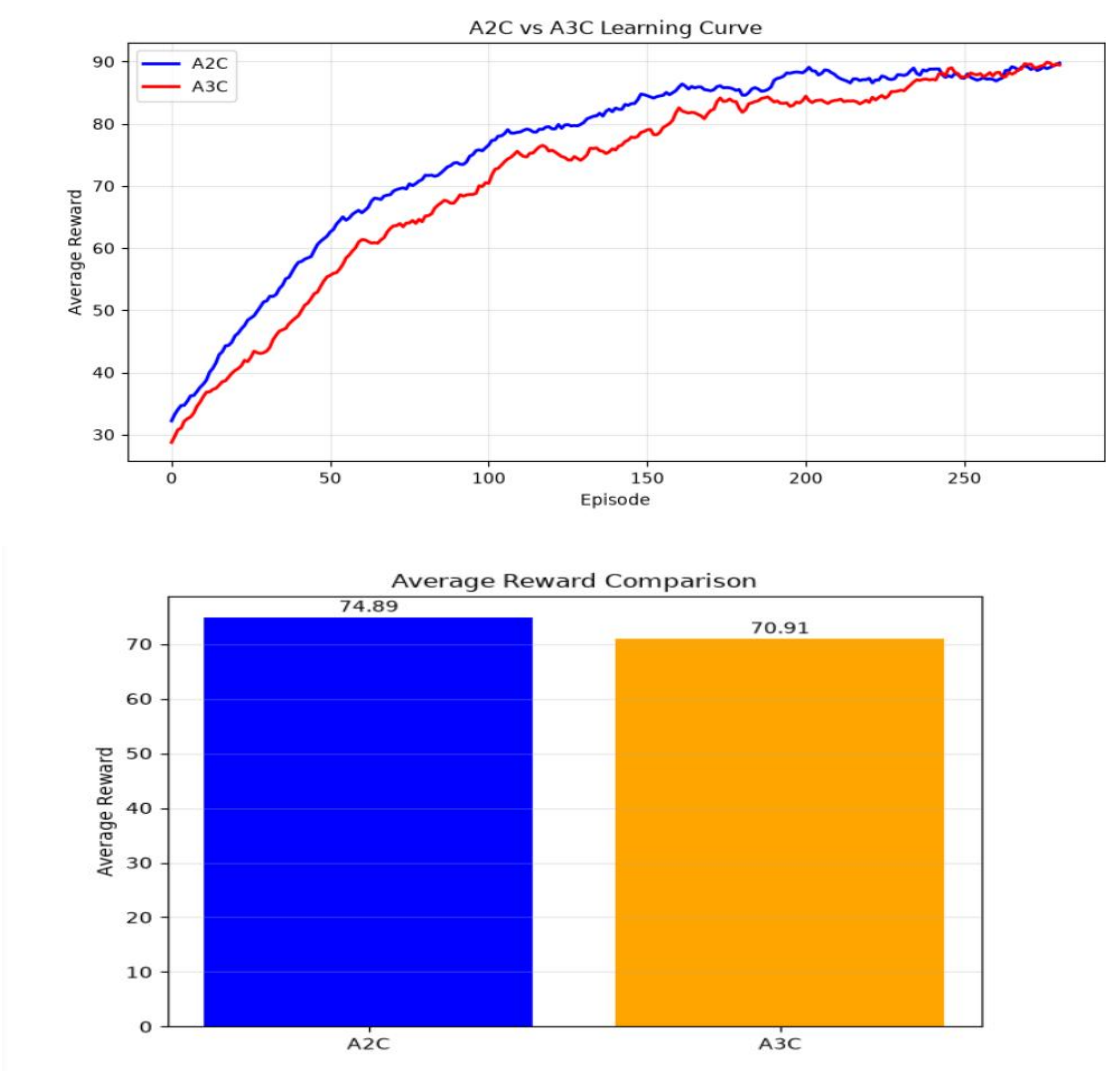
```
=====
                        A2C vs A3C SUMMARY
=====
Algorithm  Average Reward  Final Reward  Best Reward  Std Deviation
      A2C           74.89           89.41           100.0           17.53
      A3C           70.91           89.38           100.0           18.95

=====
                        COMPARISON
=====
Higher Average Reward : A2C
Better Final Performance : A2C
Highest Reward : A3C
More Stable Method : A2C

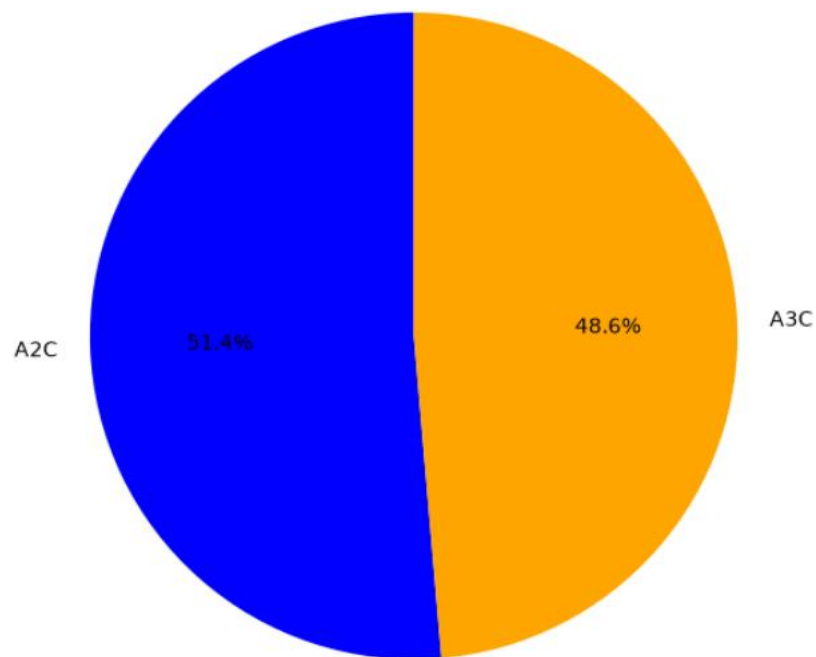
Average Reward Difference : 3.97
Final Reward Difference   : 0.03

=====
                        FINAL RESULT
=====
A2C performed better overall.
A2C Average Reward : 74.89
A3C Average Reward : 70.91
```

Visualization:



A2C vs A3C Performance Share



Reward Stability Comparison

